Python: OOP Deep-Dive & Iteration
Compilation: Yogesh Haribhau Kulkarni

OOP Deep-Dive & Iteration

Object-Oriented Programming: Concepts

Managing Larger Programs

Procedural programming is:

- Sequential code
- Conditional code (if statements)
- Repetitive code (loops)
- Store and reuse (functions)

In a very large codebase, object oriented programming is a way to arrange your code so that you can zoom into 500 lines of the code, and understand it while ignoring the other 999,500 lines of code for the moment.

Starting with Programs

- One way to think about object oriented programming is that we are separating our program into multiple "zones".
- Each "zone" contains some code and data (like a program) and has well defined interactions with the outside world and the other zones within the program.

Sample program

```
import urllib.request, urllib.parse, urllib.error
from bs4 import BeautifulSoup
import ssl

# Ignore SSL certificate errors
ctx = ssl.create_default_context()
ctx.check_hostname = False
ctx.verify_mode = ssl.CERT_NONE

url = input('Enter - ')
html = urllib.request.urlopen(url,
    context=ctx).read()
soup = BeautifulSoup(html, 'html.parser')

# Retrieve all of the anchor tags
tags = soup('a')
for tag in tags:
  print(tag.get('href', None))
```

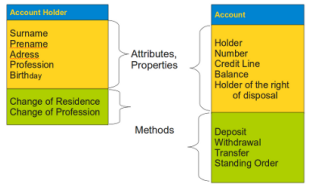
Subdividing a Problem

- One of the advantages of the object oriented approach is that it can hide complexity.
- For example, while we need to know how to use the urllib and BeautifulSoup code, we do not need to know how those libraries work internally.
- It allows us to focus on the part of the problem we need to solve and ignore the other parts of the program.

What's an *object*?

A Python object is a bundle of variables and functions.

- What variable names and functions comprise an object is defined by the object's *class*.
- From one class specification, many objects can be *instantiated*. Different instances can assign different values to the object variables.
- Variables and functions in an instance are collectively called *instance attributes*; functions are also termed *instance methods*.



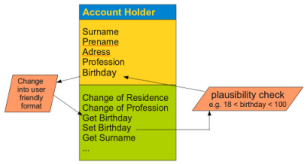
Intuition: Classes vs. Objects

Intuition

Think of a *class* as a blueprint (e.g., for a house) and an *object* as one actual house built from that blueprint. Many houses (objects) can be built from the same blueprint (class), each with its own address, paint color, and furniture (instance attributes) – but all sharing the same floor plan (methods).

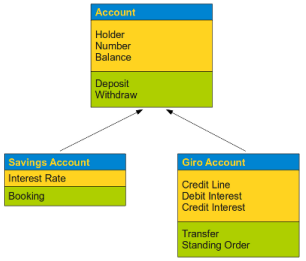
Encapsulation of Data

- Encapsulation is the mechanism for restricting the access to some of an object's components, this means that the internal representation of an object can't be seen from outside of the objects definition.
- Access to this data is typically only achieved through methods



Inheritance

- A class can inherit attributes and behaviour (methods) from other classes, called super-classes.
- Inheritance: to create new classes by using existing classes.
- New ones can both be created by extending and by restricting the existing classes.



Using Objects

Python provides us with many built-in objects. Below, the first line constructs an object of type `list`; the second and third lines call the `append()` method; the fourth line calls `sort()`; the fifth line retrieves the item at position 0; and the sixth line calls `__getitem__()` on the list directly, with a parameter of zero:

```
stuff = list()
stuff.append('python')
stuff.append('chuck')
stuff.sort()
print (stuff[0])
print (stuff.__getitem__(0))
print (list.__getitem__(stuff,0))
```

Example

Import the `date` class from the standard library module `datetime`. To instantiate an object, call the class name like a function:

```
>>> from datetime import date
>>> dt1 = date(2012, 9, 28)
>>> dt2 = date(2012, 10, 1)
```

Example

```
>>> dir(dt1)
['__add__', '__class__', 'ctime', 'day',
'fromordinal', 'fromtimestamp', 'isocalendar',
'isoformat', 'isoweekday', 'max', 'min', 'month',
'replace', 'resolution', 'strftime', 'timetuple',
'today', 'toordinal', 'weekday', 'year']
```

The `dir` function can list all objects attributes. Note there is no distinction between instance variables and methods! Access to object attributes is done by suffixing the instance name with the attribute name, separated by a dot ".".

```
>>> dt1.day
28
>>> dt1.month
9
>>> dt1.year
2012
```

Starting with OOP

When the party method is called, the first parameter (`self`) points to the particular instance of the `PartyAnimal` object that `party` is called from within – note the line `self.x = self.x + 1` inside the method:

```
class PartyAnimal:
  x = 0
  def party(self) :
    self.x = self.x + 1
    print("So far",self.x)

an = PartyAnimal()
an.party()
an.party()
an.party()
PartyAnimal.party(an)
```

Starting with OOP

Following line is another way to call the party method within the an object: `PartyAnimal.party(an)` When the program executes, it produces the following output:

```
So far 1
So far 2
So far 3
So far 4
```

The `self` argument

Every method of a Python object always has `self` as first argument.

However, you do not specify it when calling a method: it's automatically inserted by Python:

```
>>> class ShowSelf(object):
...   def show(self):
...     print(self)
...
>>> x = ShowSelf() # construct instance
>>> x.show() # 'self' automatically inserted!
<__main__.ShowSelf object at 0x299e150>
```

The `self` name is a reference to the object instance itself. You *need to* use `self` when accessing methods or attributes of this instance.

Intuition: Why `self`?

Intuition

Read `self.x` as "*my* x" – it tells Python which object's data to use. Without `self`, a method would have no way to know *which* instance called it, since the same method body is shared by every instance of the class.

Member functions

There are different types of methods/functions – instance, class, and static – shown below. (In Python 3, all classes are "new-style" by default; explicitly inheriting from `object`, e.g. `class MyClass(object):`, is optional but still common.)

```
class MyClass:
    def method(self):
        return 'instance method called', self

    @classmethod
    def classmethod(cls):
        return 'class method called', cls

    @staticmethod
    def staticmethod():
        return 'static method called'
```

Member functions: Instance Methods

Through the `self` parameter, instance methods can freely access attributes and other methods on the same object:

```
class MyClass:
    def method(self):
        return 'instance method called', self
```

Member functions: Class Methods

Marked with a `@classmethod` decorator, a class method takes a `cls` parameter that points to the class itself – not the object instance – when called:

- Because it only has access to `cls`, a class method can't modify object instance state – that would require `self`.
- However, class methods can still modify class state that applies across all instances of the class.

```
class MyClass:
    @classmethod
    def classmethod(cls):
        return 'class method called', cls
```

Member functions: Static Methods

Marked with a `@staticmethod` decorator, this type of method takes neither a `self` nor a `cls` parameter (though it's free to accept other parameters):

- Therefore a static method can neither modify object state nor class state. Static methods are restricted in what data they can access - and they're primarily a way to namespace your methods.

```
class MyClass:
    @staticmethod
    def staticmethod():
        return 'static method called'
```

Intuition: Instance vs. Class vs. Static Methods

Intuition

Picture a workshop: an *instance method* works on one specific project on the bench (`self`); a *class method* works on the workshop's shared rulebook (`cls`), affecting every project; a *static method* is just a tool kept in the workshop's toolbox – useful and related to the workshop, but it doesn't touch any specific project or the rulebook at all.

Member functions: Instance Methods

Calling `obj.method()` confirms that the instance method has access to the object instance (printed as "MyClass object") via the `self` argument. When the method is called, Python replaces the `self` argument with the instance object, `obj`:

```
>>> obj = MyClass()
>>> obj.method()
('instance method called', <__main__.MyClass
    object at 0x101a2f4c8>)
```

Member functions: Instance Methods

We could ignore the syntactic sugar of the dot-call syntax (obj.method()) and pass the instance object manually to get the same result:

```
>>> MyClass.method(obj)
('instance method called', <__main__.MyClass
    object at 0x101a2f4c8>)
```

Member functions: Class Methods

- Calling `classmethod()` shows it doesn't have access to a "MyClass object" instance, but only to the class `MyClass` itself, representing the class as an object (everything in Python is an object, even classes themselves).
- Naming these parameters `self` and `cls` is just a convention – you could just as easily name them `the_object` and `the_class` and get the same result.

```
>>> obj.classmethod()
('class method called', <class '__main__.MyClass'>)
```

Member functions: Static Methods

- It may be surprising that a static method can be called on an object instance at all – behind the scenes, Python simply skips passing `self` or `cls` when a static method is invoked via dot syntax.
- This confirms that static methods can neither access the object instance state nor the class state – they work like regular functions that happen to live in the class's (and every instance's) namespace.

```
>>> obj.staticmethod()
'static method called'
```

Member functions: Cases

When we attempt to call these methods on the class itself - without creating an object instance beforehand, `classmethod()` and `staticmethod()` still work, but `method()` fails: there is no instance for Python to populate the `self` argument with.

```
>>> MyClass.classmethod()
('class method called', <class '__main__.MyClass'>)

>>> MyClass.staticmethod()
'static method called'

>>> MyClass.method()
TypeError: method() missing 1 required
    positional argument: 'self'
```

Member functions: Why Static methods?

Rather than computing the area directly inside `area()`, the well-known circle-area formula is factored out into a separate `circle_area()` static method:

```
import math

class Pizza:
    def __init__(self, radius, ingredients):
        self.radius = radius
        self.ingredients = ingredients

    def __repr__(self):
        return (f'Pizza({self.radius!r}, '
                f'{self.ingredients!r})')

    def area(self):
        return self.circle_area(self.radius)

    @staticmethod
    def circle_area(r):
        return r ** 2 * math.pi
```

Member functions: Why Static methods?

- As don't take a cls or self argument. That's a big limitation- but it's also a great signal to show that a particular method is independent from everything else around it.
- Now, why is that useful?
- Flagging a method as a static method is not just a hint that a method won't modify class or instance state - this restriction is also enforced by the Python runtime.
- Techniques like that allow you to communicate clearly the intention.

Member functions: Properties

Getters and setters, using the `@property` decorator. The instance variable is marked private by prefixing it with an underscore; note that the instance variable and the property must have different names, or we get infinite recursion:

```
class TestProperty(object):
    def __init__(self, description):
        self._description = description
    @property
    def description(self):
        print('getting description')
        return self._description
    @description.setter
    def description(self, description):
        print('setting description')
        self._description = description
```

Member function: __init__

- "__init__" is a reseved method in python classes.
- Python doesn't have explicit constructors like C++ or Java, but the __init__() method in Python is something similar, though it is strictly speaking not a constructor.
- It behaves in many ways like a constructor, e.g. it is the first code which is executed, when a new instance of a class is created.

```
class Car(object):
  def __init__(self, model, color, company,
     speed_limit):
    self.color = color
    self.company = company
    self.speed_limit = speed_limit
    self.model = model
```

Member function: Destructor

- There is no "real" destructor, but something similiar, i.e. the method __del__.
- It is called when the instance is about to be destroyed.
- If a base class has a __del__() method, the derived class's __del__() method, if any, must explicitly call it to ensure proper deletion of the base class part of the instance.

```
class Greeting:
    def __init__(self, name):
        self.name = name
    def __del__(self):
        print("Destructor started")
    def SayHello(self):
        print("Hello", self.name)
```

Member function: Example Construction Destruction

- Using this class shows that `del` doesn't directly call the __del__() method.
- The destructor is not called when we delete `x1`: `del` only decrements the reference count for that object by one.
- Only once the reference count reaches zero – here, after `x2` (the last remaining reference) is also deleted – is the destructor actually called.

```
>>> from hello_class import Greeting
>>> x1 = Greeting("Guido")
>>> x2 = x1
>>> del x1
>>> del x2
Destructor started
```

No access control

There are no "public", "private" and 'protected". qualifiers for object attributes.
***Any* code can create/read/overwrite/delete *any* attribute on *any* object.**
There are *conventions*, though:

- "protected" attributes: _name_
- "private" attributes: __name__

(But again, note that this is not *enforced* by the system in any way.)

No overloading

- Python does not allow overloading of functions.
- Any function.
- Hence, no overloading of constructors.
- So: **a class has one and only one constructor.**

Constructor chaining

When a class is instantiated, Python only calls the first constructor it can find in the class inheritance call-chain.
If you need to call a parent constructor, you need to do it *explicitly*:

```
class Application(Task):
  def __init__(self, ...):
    # do Application-specific stuff here
    Task.__init__(self, ...)
    # some more Application-specific stuff
```

Calling a superclass constructor is optional, and it can happen anywhere in the init method body.

Classes as Types

All variables have a type. And we can use the built-in dir function to examine the capabilities of a variable. We can use type and dir with the classes that we create.

```
class PartyAnimal:
  x = 0
  def party(self) :
    self.x = self.x + 1
    print("So far",self.x)

an = PartyAnimal()
print ("Type", type(an))
print ("Dir ", dir(an))
print ("Type", type(an.x))
print ("Type", type(an.party))
```

Object Life-cycle

As our objects become more complex, we need to take some action within the object to set things up as the object is being constructed and possibly clean things up as the object is being discarded.

```
class PartyAnimal:
  x = 0
  def __init__(self):
    print('I am constructed')

  def party(self) :
    self.x = self.x + 1
    print('So far',self.x)

  def __del__(self):
    print('I am destructed', self.x)

an = PartyAnimal()
an.party()
an.party()
an = 42
print('an contains',an)
```

Many Instances

When we are making multiple objects from our class, we might want to set up different initial values for each of the objects.

```
class PartyAnimal:
  x = 0
  name = ''
  def __init__(self, nam):
    self.name = nam
    print(self.name,'constructed')
  def party(self) :
    self.x = self.x + 1
    print(self.name,'party count',self.x)

s = PartyAnimal('Sally')
s.party()
j = PartyAnimal('Jim')
j.party()
s.party()
```

Inheritance

Ability to create a new class by extending an existing class. When extending a class, we call the original class the 'parent class' (object) and the new class as the 'child class' (PartyAnimal).

```
class PartyAnimal(object):
  x = 0
  name = ''
  def __init__(self, nam):
    self.name = nam
    print(self.name,'constructed')

  def party(self) :
    self.x = self.x + 1
    print(self.name,'party count',self.x)
```

Child Class

We can 'import' the PartyAnimal class in a new file and extend it as follows:

```
from party import PartyAnimal

class CricketFan(PartyAnimal):
  points = 0

  def six(self):
    self.points = self.points + 6
    self.party()
    print(self.name,"points",self.points)

s = PartyAnimal("Sally")
s.party()
j = CricketFan("Jim")
j.party()
j.six()
print(dir(j))
```

Inheritance: Try

```
class Vehicle:
    def __init__(self, name, color):
        self.__name = name      # __name is
     private to Vehicle class
        self.__color = color
    def getColor(self):         # getColor()
     function is accessible to class Car
        return self.__color
    def setColor(self, color):  # setColor is
     accessible outside the class
        self.__color = color
    def getName(self):          # getName() is
     accessible outside the class
        return self.__name
```

Inheritance: Try

```
class Car(Vehicle):
    def __init__(self, name, color, model):
        # call parent constructor to set name and
     color
        super().__init__(name, color)
        self.__model = model
    def getDescription(self):
        return self.getName() + self.__model + "
     in " + self.getColor() + " color"

# in method getDescrition we are able to call
     getName(), getColor() because they are
# accessible to child class through inheritance
c = Car("Ford Mustang", "red", "GT350")
print(c.getDescription())
print(c.getName()) # car has no method getName()
     but it is accessible through class Vehicle
```

Multiple Inheritance

A class can inherit from more than one class.

```
class MySuperClass1():

    def method_super1(self):
        print("method_super1 method called")

class MySuperClass2():

    def method_super2(self):
        print("method_super2 method called")

class ChildClass(MySuperClass1, MySuperClass2):
    def child_method(self):
        print("child method")

c = ChildClass()
c.method_super1()
c.method_super2()
```

Overriding methods

To override a method in the base class, sub class needs to define a method of same signature. (i.e same method name and same number of parameters as method in base class). Try commenting out `m1()` in class `B` afterward – the version from base class `A` will run instead:

```
class A():
    def __init__(self):
        self.__x = 1
    def m1(self):
        print("m1 from A")

class B(A):
    def __init__(self):
        self.__y = 1
    def m1(self):
        print("m1 from B")

c = B()
c.m1() # m1 from B
```

isinstance() function

Used to determine whether the object is an instance of the class or not.

```
>>> isinstance(1, int)
True

>>> isinstance(1.2, int)
False

>>> isinstance([1,2,3,4], list)
True
```

Inspection

- Learn what the type of an object is – Example: `type(obj)`
- Learn what attributes an object has and what it's capabilities are – Example: `dir(obj)`
- Get help on a class or an object – Example: `help(obj)`
- In Ipython: `In [49]: a.upper?`

Exercise

- Define a class which has at least two methods:
- getString: to get a string from console input
- printString: to print the string in upper case.
- Also please include simple test function to test the class methods.
- Hints: Use __init__ method to construct some parameters

Solution

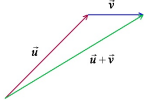
```
class InputOutString(object):
    def __init__(self):
        self.s = ""

    def getString(self):
        self.s = input()

    def printString(self):
        print(self.s.upper())

strObj = InputOutString()
strObj.getString()
strObj.printString()
```

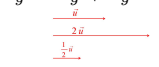
Object Oriented Programming: Examples

Recall: *What is a 2D vector?*

- A *2D* vector is an element of the vector space $\mathbb{R}^2$.
- Every *2D* vector $\mathbf{u}$ is completely described by a pair of real coordinates $\langle u_x, u_y \rangle$.
- Note: It is not a point in space. It gives Direction, like a movement recipe.
- When added to a point, results into a transformed point.

Two operations are defined on vectors:
vector addition: if $\mathbf{w} = \mathbf{u} + \mathbf{v}$, then $w_x = u_x + v_x$ and $w_y = u_y + v_y$.





scalar multiplication: if $\mathbf{v} = \alpha \cdot \mathbf{u}$ with $\alpha \in \mathbb{R}$, then $v_x = \alpha \cdot u_x$ and $v_y = \alpha \cdot u_y$.

A *2D vector* in Python

```
class Vector(object):

  def __init__(self, x, y):
    self.x = x
    self.y = y

  def add(self, other):
    return Vector(self.x+other.x,
                  self.y+other.y)

  def mul(self, scalar):
    return Vector(scalar*self.x, scalar*self.y)

  def show(self):
    return ('<{},{}>'.format(self.x, self.y))
```

DocString

Classes can have docstrings. The content of a class docstring will be shown as help text for that class.

```
class Vector(object):
  """A 2D Vector."""
  def __init__(self, x, y):
    self.x = x
    self.y = y
  def add(self, other):
    return Vector(self.x+other.x,
                  self.y+other.y)
  def mul(self, scalar):
    return Vector(scalar*self.x, scalar*self.y)
  def show(self):
    return ('<{},{}>'.format(self.x, self.y))
```

Member functions

The **def** keyword introduces a method definition. Every method *must* have at least one argument, named **self**.

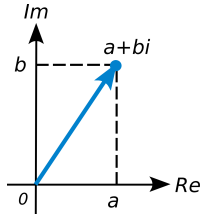
```
class Vector(object):
  """A 2D Vector."""
  def __init__(self, x, y):
    self.x = x
    self.y = y
  def add(self, other):
    return Vector(self.x+other.x,
                  self.y+other.y)
  def mul(self, scalar):
    return Vector(scalar*self.x, scalar*self.y)
  def show(self):
    return ('<{},{}>'.format(self.x, self.y))
```

Exercises

- Add a new method `norm` the `Vector` class: if `v` is an instance of class `Vector`, then calling `v.norm()` returns the norm $\sqrt{v_x^2 + v_y^2}$ of the associated vector.
- Add a new method `unit` to the `Vector` class: if `v` is an instance of class `Vector`, then calling `v.unit()` returns the vector `u` having the same direction as `v` but norm 1.

Recall: *What is a complex number?*

A complex number $\mathbf{z}$ has the form $\mathbf{z} = z_1 + z_2 i$, where z_1 and z_2 are real numbers; z_1 is called the real part and z_2 the imaginary part.



The set of complex numbers is a field with the two operations:

- *addition:* if $\mathbf{z} = \mathbf{x} + \mathbf{y}$, where $\mathbf{x} = x_1 + x_2 i$ and $\mathbf{y} = y_1 + y_2 i$ then: $z_1 = x_1 + y_1$, and $z_2 = x_2 + y_2$.
- *multiplication:* if $\mathbf{z} = \mathbf{x} \cdot \mathbf{y}$, then: $z_1 = x_1 \cdot y_1 - x_2 \cdot y_2$, and $z_2 = x_2 \cdot y_1 + x_1 \cdot y_2$.

Naive complex numbers in Python

Python object that implements a complex number.

```
class ComplexNum(object):
  "A complex number z = z1 + z2*i."
  def __init__(self, z1, z2):
    self.re = z1
    self.im = z2
  def __add__(self, other):
    return ComplexNum(
      self.re+other.re, self.im+other.im)
  def __mul__(self, other):
    return ComplexNum(
      self.re*other.re - self.im*other.im,
      self.re*other.im + self.im*other.re)
  def __str__(self):
    return ("%g + %gi" % (self.re, self.im))
  def __eq__(self, other):
    return (self.re == other.re) \
           and (self.im == other.im)
```

What does `ComplexNum` do?

We can create complex numbers by initializing them with the real and imaginary part:

```
>>> u = ComplexNum(1,0)
>>> i = ComplexNum(0,1)
>>> z = ComplexNum(1,1)
```

The str method provides the familiar representation:

```
>>> print(u)
1 + 0i
>>> print(i)
0 + 1i
>>> print(z)
1 + 1i
```

The add method makes the "`+`" operator work:

```
>>> w = u + i
>>> print(w)
1 + 1i
```

The mul method makes the "`*`" operator work:

```
>>> m = i*i
>>> print(m)
-1 + 0i
```

Exercises

- Add a to_power method to class `ComplexNum`, that takes an integer number as argument and returns the complex number raised to that power.
- Verify that z.to_power(1), z.to_power(2) and z.to_power(3) yield the same result as z, z*z and z*z*z.
- Example:

```
>>> z = ComplexNum(1,1)
>>> w = z.to_power(3)
>>> print(w)
-2 + 2i
```

Exercises

Modify the mul method of class `ComplexNum`:

- If second argument `other` is an object of class `int` (integer) or `float` (floating-point number) then return the result of scalar multiplication by that number.
- If second argument `other` is an object of class `ComplexNum`, then return the result of complex multiplication by that number.

`Vector` vs `ComplexNum`

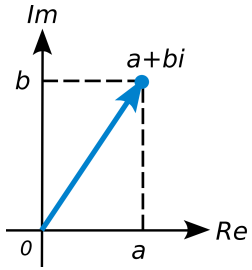
There's a lot of similarities in the two classes!
How can we share code between the two?

```
class Vector(object):
  def __init__(self, x, y):
    self.x = x
    self.y = y
  def __eq__(self, other):
    return (self.x == other.x) \
           and (self.y == other.y)
  def __add__(self, other):
    return Vector(
      self.x+other.x, self.y+other.y)
  def __mul__(self, scalar):
    return Vector(
      scalar*self.x,
      scalar*self.y)
  def __str__(self):
    return ("<%g,%g>" % (self.x, self.y))
```

```
class ComplexNum(object):
  def __init__(self, z1, z2):
     self.re = z1
     self.im = z2
  def __eq__(self, other):
      return (self.re == other.re) \
          and (self.im == other.im)
  def __add__(self, other):
      return ComplexNum(
      self.re+other.re, self.im+other.im)
  def __mul__(self, other):
    return ComplexNum(
      self.re*other.re - self.im*other.im,
      self.re*other.im + self.im*other.re)
  def __str__(self):
     return ("%g + %gi" % (self.re, self.im))
```

Vectors and *Complex Numbers*

Complex Numbers are in $1-1$ correspondence with points in the real plane $\mathbb{R}^2$: the set $\mathbb{C}$ of Complex Numbers is the set of real *2D* vectors.



- So we can define the class of Complex Numbers as the class of Vectors augmented with a multiplication operation.
- So: a Complex Number is a Vector plus some behavior.

Inheritance, I

```
class Vector(object):

  def __init__(self, x, y):
    self.x = x
    self.y = y

  def __eq__(self, other):
    return (self.x == other.x) \
           and (self.y == other.y)

  def __add__(self, other):
    return self.__class__(
      self.x+other.x, self.y+other.y)

  def __mul__(self, scalar):
    return Vector(
      scalar*self.x,
      scalar*self.y)

  def __str__(self):
    return ("<%g,%g>" % (self.x, self.y))
```

- `class ComplexNum` is a *child/descendant/subclass* of class `Vector`.
- `class Vector` is a *parent/ancestor/superclass* of class `ComplexNum`.

```
class ComplexNum(Vector):

  def __mul__(self, other):
    return ComplexNum(
      self.x*other.x - self.y*other.y,
      self.x*other.y + self.y*other.x)

  def __str__(self):
    return ("%g + %gi" % (self.x, self.y))
```

Inheritance, II

```
class Vector(object):

  def __init__(self, x, y):
    self.x = x
    self.y = y

  def __eq__(self, other):
    return (self.x == other.x) \
           and (self.y == other.y)

  def __add__(self, other):
    return self.__class__(
      self.x+other.x, self.y+other.y)

  def __mul__(self, scalar):
    return Vector(
      scalar*self.x,
      scalar*self.y)

  def __str__(self):
    return ("<%g,%g>" % (self.x, self.y))
```

All methods defined in class `Vector` are automatically defined *(inherited)* in class `ComplexNum`.

```
class ComplexNum(Vector):

  def __mul__(self, other):
    return ComplexNum(
      self.x*other.x - self.y*other.y,
      self.x*other.y + self.y*other.x)

  def __str__(self):
    return ("%g + %gi" % (self.x, self.y))
```

Inheritance, III

```
class Vector(object):

  def __init__(self, x, y):
    self.x = x
    self.y = y

  def __eq__(self, other):
    return (self.x == other.x) \
           and (self.y == other.y)

  def __add__(self, other):
    return self.__class__(
      self.x+other.x, self.y+other.y)

  def __mul__(self, scalar):
    return Vector(
      scalar*self.x,
      scalar*self.y)

  def __str__(self):
    return ("<%g,%g>" % (self.x, self.y))
```

Methods mul and str are defined in both classes: instances of a class use the definition from that class.

We say that `ComplexNum` *overrides* those methods from `Vector`.

```
class ComplexNum(Vector):

  def __mul__(self, other):
    return ComplexNum(
      self.x*other.x - self.y*other.y,
      self.x*other.y + self.y*other.x)

  def __str__(self):
    return ("%g + %gi" % (self.x, self.y))
```

Method chaining

Actually, init is not special in this regard.

When any instance method is called, Python only calls the first constructor it can find in the class inheritance call-chain.

You can always call a superclass method by prefixing it with the superclass name and explicitly writing "`self`" as the first argument:

```
class ComplexNum(Vector):
  # ...
  def print_as_vector(self):
    return Vector.__str__(self)
```

Inheritance, IV

Take a look at this Python interaction – what do you expect `print(u)` to show? (Spoiler: not a `ComplexNum`.)

```
>>> z = ComplexNum(1,0)
>>> print(z)
1 + i0
>>> w = ComplexNum(0,1)
>>> print(w)
0 + i1
>>> u = z + w
>>> print(u)
```

Inheritance, V

The answer is in the code:

```
class Vector(object):
  # ...
  def __add__(self, other):
    return Vector(self.x+other.x, self.y+other.y)
```

The add method returns a new Vector instance, even when called from a ComplexNum !

Inheritance, VI

Correct code:

```
class Vector(object):
  # ...
  def __add__(self, other):
    return self.__class__(self.x+other.x,
     self.y+other.y)
```

Use the class of the actual instance that's passed, instead of hard-coding a class name.

Polymorphism, I

The ability to implement different behavior for the same method/operator in different classes is called *polymorphism*.

The multiplication operator "`*`" on instances of the Vector class works as *scalar multiplication*:

```
>>> v = Vector(1,0)
>>> print (v * 3)
<3,0>
```

The same operator "`*`" on instances of the ComplexNum class works as *complex multiplication*:

```
>>> z = ComplexNum(1,1)
>>> w = ComplexNum(2,0)
>>> print (z * w)
2 + i2
```

Polymorphism, II

You may observe that an integer is (in particular) a complex number, still we cannot multiply a `ComplexNum` instance by an integer number. Our code for `mul` implicitly assumes that argument `other` has attributes `x` and `y`, and integers do not:

```
>>> print (z * 3)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "vector_and_complexnum.py", line 20, in
    __mul__
    self.x*other.x - self.y*other.y,
AttributeError: 'int' object has no attribute 'x'
```

Modern Python: `@dataclass`

Every class in this deck (`Car`, `Greeting`, `Vector`, `ComplexNum`, ...) hand-wrote its own `__init__`. Since Python 3.7, the `@dataclass` decorator generates `__init__`, `__repr__`, and `__eq__` automatically from type-annotated fields – this is purely a convenience for simple "data-holding" classes, and you can still add your own methods exactly as before:

```
from dataclasses import dataclass

@dataclass
class Vector:
    x: float
    y: float

    def add(self, other):
        return Vector(self.x + other.x, self.y +
     other.y)

>>> v = Vector(1, 0)
>>> print(v)
Vector(x=1, y=0)
```

Recall: Object Oriented Programming

- A Python object is a bundle of variables and functions.
- Defined by the object's *class*.
- From class, many objects can be *instantiated*.
- Different instances can assign different values to the object variables.

Quick Check: Object-Oriented Programming

Think About It

Two classes, `Vector` and `ComplexNum(Vector)`, both define `__mul__` and `__str__`, but only `Vector` defines `__add__`. If `z` and `w` are both `ComplexNum` instances, what type is `z + w`, and why does that matter?

Answer

`z + w` is a plain `Vector`, not a `ComplexNum` – because `__add__` is inherited unchanged from `Vector`, and its body hard-codes `return Vector(...)` rather than using `self.__class__(...)`. This is exactly the bug seen in "Inheritance, IV/V": a method inherited by a subclass still constructs instances of the *class it was written in*, unless it's written to respect the actual instance's class.

Special Methods

What we shall see in this part

How to define custom behavior for Python's standard operators and functions on user-defined objects.
(Technically called "operator overloading.")

Python's special methods

- Names that start and end with two underscores e.g., `__init__` have a special significance in Python.
- However, there is nothing special in the way we define those methods: the syntax is the same as for any other method.
- Most of the special methods directly map to Python's operators (e.g., "`+`", "`==`" or "`in`").

First Example (1/2)

Let us rename the `show` method to `__str__`.

```
class Vector(object):
  """A 2D Vector."""
  def __init__(self, x, y):
    self.x = x
    self.y = y

  def add(self, other):
    return Vector(self.x+other.x,
                  self.y+other.y)

  def mul(self, scalar):
    return Vector(scalar*self.x, scalar*self.y)

  def __str__(self):
    return ("<%g,%g>" % (self.x, self.y))
```

First Example (2/2)

```
>>> from vector2 import Vector
>>> v = Vector(0,1)
>>> print(v)
<0,1>
```

- Now Python's built-in `print` behaves like the `show` method did!

- Actually, `print` uses Python's built-in function `str()` to convert an object to a string, and then prints this string.
- By defining the `__str__` method, we override the default behavior of Python's `str()` for objects of class `Vector`.

Second Example: equality testing (1/3)

- Can we test two instanced of class `Vector` for equality?

```
>>> from vector2 import Vector
>>> v1 = Vector(0,1)
>>> v2 = Vector(0,1)
>>> v1 == v2
False
```

- Python does not know how to test if two user defined objects are equal.
- By default "`==`" behaves like the "`is`" operator on user-defined classes
- Two user-defined objects are considered equal if and only if they are the same object.
- This can be changed by adding a `__eq__` method.

Equality *vs* identity

- The `is` operator returns `True` if two names refer to the same instance; the `==` operator compares the *values* of two objects.[1]
- Note that two instances may be equal in any respect yet be different instances: *equality is not identity!*

```
>>> dt4 = date(2012,9,28)
>>> dt5 = date(2012,9,28)
>>> dt4 == dt5
True
>>> dt4 is dt5
False
```

Second Example: equality testing (2/3)

Let us add an eq method.

```
class Vector(object):
  """A 2D Vector."""
  def __init__(self, x, y):
    self.x = x
    self.y = y
  # (code omitted)
  def __str__(self):
    return ("<%g,%g>" % (self.x, self.y))
  \textbf{def} __eq__(\textbf{self}, other):
    return (self.x == other.x) and (self.y ==
     other.y)
```

Second Example: equality testing (3/3)

```
>>> from vector3 import Vector
>>> v1 = Vector(0,1)
>>> v2 = Vector(0,1)
>>> v1 == v2
True
```

By defining the eq method, we define the behavior of Python's equality test == for objects of class `Vector`.

3rd Example: vector addition (1/3)

The add special method defines the behavior of the "`+`" operator.
Let's just rename `add` to `__add__`.

```
class Vector(object):
  """A 2D Vector."""
  def __init__(self, x, y):
    self.x = x
    self.y = y
  def __str__(self):
    return ("<%g,%g>" % (self.x, self.y))
  def __eq__(self, other):
    return (self.x == other.x) and (self.y ==
     other.y)
  \textbf{def} __add__(\textbf{self}, other):
    return Vector(self.x+other.x, self.y+other.y)
```

3rd Example: vector addition (2/3)

Now vector addition works with the usual "`+`" operator:

```
>>> from vector4 import Vector
>>> v1 = Vector(1,0)
>>> v2 = Vector(0,1)
>>> print (v1 + v2)
<1,1>
```

3rd Example: vector addition (3/3)

What if we add inhomogeneous objects, e.g., a vector and a number?

```
>>> from vector4 import Vector
>>> v1 = Vector(1,0)
>>> print (v1 + 5.0)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "vector4.py", line 14, in __add__
    return Vector(self.x+other.x, self.y+other.y)
AttributeError: 'float' object has no attribute 'x'
```

In this case, an error is the correct behavior: a vector can only be summed to another vector.

We shall see in the next part that sometimes it makes sense to allow in homogeneous operations, and how to implement them.

4th Example: vector multiplication (1/3)

The mul special method defines the behavior of the "`*`" operator.

```
class Vector(object):
  """A 2D Vector."""
  def __init__(self, x, y):
    self.x = x
    self.y = y
  def __str__(self):
    return ("<%g,%g>" % (self.x, self.y))
  def __eq__(self, other):
    return (self.x == other.x) and (self.y ==
     other.y)
  def __add__(self, other):
    return Vector(self.x+other.x, self.y+other.y)
  \textbf{def} __mul__(\textbf{self}, scalar):
    return Vector(scalar*self.x, scalar*self.y)
```

4th Example: vector multiplication (2/3)

Now vector multiplication works with the usual "`*`" operator:

```
>>> from vector5 import Vector
>>> v1 = Vector(1,2)
>>> v1 * 3 == Vector(3, 6)
>>> print (v1 * 2)
<2,4>
```

[1] A class can define how exactly the `==` operator should carry out the comparison.

4th Example: vector multiplication (3/3)

- Note that:

```
>>> from vector5 import Vector
>>> v1 = Vector(1,2)
>>> 3 * v1
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: unsupported operand type(s) for *:
     'int' and 'Vector'
```

- Order matters! Our `__mul__` method requires a `Vector` instance first, and a number second.
- The operation with swapped operands is called `__rmul__`, but treating this in detail would take us too far!
- Take-home message: **Operations defined with special methods are not automatically commutative, transitive or any other property you normally associate with the operators +, *, etc.**

Further reading

- A good and readable introduction to Python's special methods:

  http://www.rafekettler.com/magicmethods.html

- Special methods hook directly into Python's syntax. They can enhance readability or make code completely obscure. Use them sparingly, and remember: *with great power comes great responsibility!*

Iterators

Usage of "for" loop

Looping over a list:

```
>>> for i in [1, 2, 3, 4]:
...     print(i)
```

With a string, it loops over its characters:

```
>>> for c in "python":
...     print(c)
```

Usage of "for" loop

With a dictionary, it loops over its keys:

```
>>> for k in {"x": 1, "y": 2}:
...     print(k)
```

With a file, it loops over lines of the file:

```
>>> for line in open("a.txt"):
...     print(line)
```

Iterables

- So there are many types of objects which can be used with a for loop.
- These are called iterable objects.
- There are many functions which consume these iterables:

```
>>> ",".join(["a", "b", "c"])
'a,b,c'
>>> ",".join({"x": 1, "y": 2})
'x,y'
>>> list("python")
['p', 'y', 't', 'h', 'o', 'n']
>>> list({"x": 1, "y": 2})
['x', 'y']
```

Iteration

- The act of going over a collection
- Explicit in the form of 'for'
- Implicit in many forms: list(), tuple(), dict(), map(), reduce(), zip()
- Used with 'in'

Iteration

- An iterator is an object adhering to the iterator protocol : basically this means that it has a next method, which, when called, returns the next item in the sequence,
- and when there's nothing to return, raises the StopIteration exception.
- An iterator object allows to loop just once.
- It holds the state (position) of a single iteration, or from the other side, each loop over a sequence requires a single iterator object.
- This means that we can iterate over the same sequence more than once concurrently.
- Separating the iteration logic from the sequence allows us to have more than one way of iteration.

Iteration

- Calling the __iter__ method on a container to create an iterator object is the most straightforward way to get hold of an iterator.
- The iter function does that for us, saving a few keystrokes.
- When used in a loop, StopIteration is swallowed and causes the loop to finish.
- But with explicit invocation, we can see that once the iterator is exhausted, accessing it raises an exception.

Iteration

- Support for iteration is pervasive in Python: all sequences and unordered containers in the standard library allow this.
- The concept is also stretched to other things: e.g. file objects support iteration over lines.
- The file is an iterator itself and it's __iter__ method doesn't create a separate object: only a single thread of sequential access is allowed.

Iteration Protocol

- Protocol of 2 methods:
  - `__iter__()`: returns the iterator object itself
  - `__next__()`: returns the next value; raises `StopIteration` when exhausted
- Call via the built-in: `next(obj)` invokes `obj.__next__()`
- Explicit iterator creation: `iter(obj)` invokes `obj.__iter__()`

Intuition: The Iterator Protocol

Intuition

`iter()` hands you a ticket-dispenser (the iterator) for a queue; `next()` advances the dispenser one ticket at a time. Once the queue is empty, the dispenser doesn't hand you a blank ticket – it tells you it's out (`StopIteration`), so any loop built on top of it knows exactly when to stop.

Iteration Protocol

The built-in function `iter()` takes an iterable object and returns an iterator.

```
>>> x = iter([1, 2, 3])
>>> x
<list_iterator object at 0x1004ca850>
>>> next(x)
1
```

```
>>> next(x)
2
>>> next(x)
3
```

Iteration Protocol

Each call to `next()` returns the next element. When exhausted, it raises `StopIteration`:

```
>>> next(x)
2
>>> next(x)
3
>>> next(x)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
StopIteration
```

Reading Lines

Without Iterator:

```
with open('data.txt', 'r') as f:
    while True:
        line = f.readline()
        if not line:
            break
        else:
            print(line.rstrip())
```

Reading Lines

With Iterator:

```
with open('data.txt', 'r') as f:
    for line in f:
        print(line.rstrip())
```

Iterator Class

- Iterators are implemented as classes.
- Here is an iterator that works like built-in range function.

```
class yrange:
    def __init__(self, n):
        self.i = 0
        self.n = n

    def __iter__(self):
        return self

    def __next__(self):
        if self.i < self.n:
            i = self.i
            self.i += 1
            return i
        else:
            raise StopIteration()
```

Iterator Class

- `__iter__` makes an object iterable; `__next__` returns the next value.
- The built-in `next()` function calls `__next__()` on the iterator.
- When no more elements exist, `__next__` must raise `StopIteration`.

```
>>> y = yrange(3)
>>> next(y)
0
>>> next(y)
1
>>> next(y)
2
>>> next(y)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "<stdin>", line 14, in __next__
StopIteration
```

Iteration Usage

Many built-in functions accept iterators as arguments. In this case, both the iterable and iterator are the same object – notice that __iter__ returned `self`. It need not be the case always:

```
>>> list(yrange(5))
[0, 1, 2, 3, 4]
>>> sum(yrange(5))
10
```

Iteration Usage

Iterable and Iterator can be different

```
class zrange:
    def __init__(self, n):
        self.n = n

    def __iter__(self):
        return zrange_iter(self.n)
```

Iteration Usage

Iterable and Iterator can be different

```
class zrange_iter:
    def __init__(self, n):
        self.i = 0
        self.n = n

    def __iter__(self):
        # Iterators are iterables too.
        # Adding this functions to make them so.
        return self

    def __next__(self):
        if self.i < self.n:
            i = self.i
            self.i += 1
            return i
        else:
            raise StopIteration()
```

Iteration Usage

If both iteratable and iterator are the same object, it is consumed in a single iteration.

```
>>> y = yrange(5)
>>> list(y)
[0, 1, 2, 3, 4]
>>> list(y)
[]
>>> z = zrange(5)
>>> list(z)
[0, 1, 2, 3, 4]
>>> list(z)
[0, 1, 2, 3, 4]
```

Context Managers (`with` statement)

A **context manager** manages resource setup and teardown automatically — even if an exception occurs.

```
# Without context manager
f = open('data.txt')
try:
    data = f.read()
finally:
    f.close()

# With context manager (preferred)
with open('data.txt') as f:
    data = f.read()   # file is closed
     automatically on exit
```

Implement your own using `__enter__` / `__exit__` (or `contextlib.contextmanager`):

```
from contextlib import contextmanager

@contextmanager
def timer():
    import time; t = time.time()
    yield
    print(f"Elapsed: {time.time()-t:.3f}s")

with timer():
    sum(range(10_000_000))
```

Iteration Advantages

- Cleaner code
- Iterators can work with infinite sequences
- Iterators save resources

Iteration Assignment

Write an iterator class `reverse_iter` that takes a list and iterates it from the reverse direction:

```
>>> it = reverse_iter([1, 2, 3, 4])
>>> next(it)
4
>>> next(it)
3
>>> next(it)
2
>>> next(it)
1
>>> next(it)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
StopIteration
```

Quick Check: Iterators

Think About It

`yrange` and `zrange` both support calling `list(obj)` twice in a row, but only `zrange` gives the same result both times. Why?

Answer

`yrange.__iter__` returns `self`, so the iterable and the iterator are the *same* object – once its internal state (`self.i`) reaches `self.n`, it's exhausted for good, and a second `list()` call gets nothing back. `zrange.__iter__` returns a *fresh* `zrange_iter(self.n)` every time, so each `list()` call starts a brand-new iterator with its own state – exactly the iterable-vs-iterator distinction from a few slides back.

Generators

A different Iterator

- Generators simplifies creation of iterators.
- A generator is a function that produces a sequence of results instead of a single value (sequence).

```
def yrange(n):
    i = 0
    while i < n:
        yield i
        i += 1
```

Intuition: Generators

Intuition

A generator function is like a lazy vending machine: it only produces the next item when you press the button (`next()`), rather than manufacturing and storing the whole product line up front the way a list would. That makes generators the natural fit for sequences too large – or infinite – to hold in memory all at once.

Generator Usage

Each time the yield statement is executed the function generates a new value. A generator is also an iterator, so you don't have to worry about the iterator protocol:

```
>>> y = yrange(3)
>>> y
<generator object yrange at 0x401f30>
>>> next(y)
0
>>> next(y)
1
>>> next(y)
2
>>> next(y)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
StopIteration
```

Generator Example

Counter class using a generator function and its usage in a for loop.

```
def counter_generator(low, high):
    while low <= high:
       yield low
       low += 1

>>> for i in counter_generator(5,10):
...     print(i, end=' ')
...
5 6 7 8 9 10
```

Generator Examples

- Inside the while loop when it reaches to the yield statement, the value of low is returned and the generator state is suspended.
- During the second next call the generator resumed where it freeze-ed before and then the value of low is increased by one.
- It continues with the while loop and comes to the yield statement again.

Generator Examples

- When you call an generator function it returns a *generator* object.
- If you call *dir* on this object you will find that it contains __iter__ and *__next__* methods among the other methods.

```
   >>> c = counter_generator(5,10)
   >>> dir(c)
   ['__class__', '__delattr__', '__dir__',
    '__doc__', '__eq__', '__format__',
'__ge__', '__getattribute__', '__gt__',
    '__hash__', '__init__', '__iter__',
'__le__', '__lt__', '__name__', '__ne__',
    '__new__', '__next__', '__reduce__',
'__reduce_ex__', '__repr__', '__setattr__',
    '__sizeof__', '__str__', '__subclasshook__',
'close', 'gi_code', 'gi_frame', 'gi_running',
    'send', 'throw']
```

Generator Example: Fibonacci

The first number of the sequence is 0, the second number is 1, and each subsequent number is equal to the sum of the previous two numbers of the sequence itself.

```
import time
def fib():
   a, b = 0, 1
   while True:
```

```
        yield b
        a, b = b, a + b
g = fib()
try:
    for e in g:
        print(e)
        time.sleep(1)
except KeyboardInterrupt:
    print("Calculation stopped")
```

Generator Examples

- We mostly use generators for **lazy evaluations**.
- This way generators become a good approach to work with lots of data.
- If you don't want to load all the data in the memory, you can use a generator which will pass you each piece of data at a time.
- A classic example is `os.walk()` — a generator that lazily traverses directory trees without loading the whole tree into memory.
- Using the generator implementation saves memory.

Note

- "generator" to mean the generated object
- "generator function" to mean the function that generates it.
- When a generator function is called, it returns a generator object without even beginning execution of the function.
- When next method is called for the first time, the function starts executing until it reaches yield statement.
- The yielded value is returned by the next call.

Generator Example

The following example demonstrates the interplay between `yield` and calls to `next()` on a generator object.

```
>>> def foo():
...     print("begin")
...     for i in range(3):
...         print("before yield", i)
...         yield i
...         print("after yield", i)
...     print("end")
...
>>> f = foo()   # NOTHING is printed yet
>>> next(f)
begin
before yield 0
0
```

Generator Example continued

```
>>> next(f)
after yield 0
before yield 1
1
>>> next(f)
after yield 1
before yield 2
2
>>> next(f)
after yield 2
end
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
StopIteration
>>>
```

Generator Examples

```
def integers():
    """Infinite sequence of integers."""
    i = 1
    while True:
        yield i
        i = i + 1

def squares():
    for i in integers():
        yield i * i
```

Generator Examples

```
def take(n, seq):
    """Returns first n values from the given
     sequence."""
    seq = iter(seq)
    result = []
    try:
        for i in range(n):
            result.append(next(seq))
    except StopIteration:
        pass
    return result

print(take(5, squares()))  # prints [1, 4, 9, 16,
     25]
```

Generator Expressions

- Generator Expressions are generator version of list comprehensions.
- They look like list comprehensions, but returns a generator back instead of a list.

```
>>> a = (x*x for x in range(10))
>>> a
<generator object <genexpr> at 0x401f08>
>>> sum(a)
285
```

Generator Expressions

We can use the generator expressions as arguments to various functions that consume iterators.

```
>>> sum((x*x for x in range(10)))
285
```

When there is only one argument to the calling function, the parenthesis around generator expression can be omitted.

```
>>> sum(x*x for x in range(10))
285
```

Generator Expressions

```
n = (e for e in range(50000000) if not e % 3)
i = 0
for e in n:
    print(e)
    i += 1
    if i > 100:
        break
```

Generator Expressions

- A generator expression is created with round brackets.
- Creating a list comprehension in this case would be very inefficient because the example would occupy a lot of memory unnecessarily.
- Insted of this, we create a generator expression, which generates values lazily on demand.

Generator Fun Example

- Lets say we want to find first 10 (or any n) Pythagorean triplets. A triplet (x, y, z) is called Pythagorean triplet if $x * x + y * y == z * z$.
- It is easy to solve this problem if we know till what value of z to test for. But we want to find first n Pythagorean triplets.

```
>>> pyt = ((x, y, z) for z in integers() for
    y in range(1, z) for x in range(1, y)
    if x*x + y*y == z*z)
>>> take(10, pyt)
[(3, 4, 5), (6, 8, 10), (5, 12, 13), (9, 12,
    15), (8, 15, 17), (12, 16, 20), (15,
    20, 25), (7, 24, 25), (10, 24, 26),
    (20, 21, 29)]
```

Example: Reading multiple files

Lets say we want to write a program that takes a list of filenames as arguments and prints contents of all those files, like cat command in UNIX.

```
def cat(filenames):
    for f in filenames:
        for line in open(f):
            print(line)
```

Example: Reading multiple files

Now, lets say we want to print only the line which has a particular substring, like grep command in unix.

```
def grep(pattern, filenames):
    for f in filenames:
        for line in open(f):
            if pattern in line:
                print(line)
```

Example: Reading multiple files

Both these programs have lot of code in common. It is hard to move the common part to a function. But with generators makes it possible to do it.

```
def readfiles(filenames):
    for f in filenames:
        for line in open(f):
            yield line

def grep(pattern, lines):
    return (line for line in lines if pattern in
     line)
```

`yield from`: Delegating to a Sub-generator

When a generator's job is mostly to hand off to another generator, `yield from` avoids writing a manual `for ... yield` loop, and forwards the sub-generator's return value too:

```
def readfiles(filenames):
    for f in filenames:
        with open(f) as fh:
            yield from fh   # same as: for line in
     fh: yield line

def chain_two(seq1, seq2):
    yield from seq1
    yield from seq2

>>> list(chain_two([1, 2], [3, 4]))
[1, 2, 3, 4]
```

Example: Reading multiple files

The code is much simpler now with each function doing one small thing. We can move all these functions into a separate module and reuse it in other programs:

```
def printlines(lines):
    for line in lines:
        print(line, end='')

def main(pattern, filenames):
    lines = readfiles(filenames)
    lines = grep(pattern, lines)
    printlines(lines)
```

Exercise

Define a class with a generator which can iterate the numbers, which are divisible by 7, between a given range 0 and n.

Solution

```
def putNumbers(n):
    i = 0
    while i<n:
        j=i
        i=i+1
        if j%7==0:
            yield j

for i in putNumbers(100):
    print(i)
```

Generator Assignments

- Write a program that takes one or more filenames as arguments and prints all the lines which are longer than 40 characters.
- Write a function findfiles that recursively descends the directory tree for the specified directory and generates paths of all the files in the tree.
- Write a function to compute the number of python files (.py extension) in a specified directory recursively.
- Write a function to compute the total number of lines of code in all python files in the specified directory recursively.
- Write a function to compute the total number of lines of code, ignoring empty and comment lines, in all python files in the specified directory recursively.

Itertools

The itertools module in the standard library provides lot of interesting tools to work with iterators.

`chain`: chains multiple iterators together.

```
>>> it1 = iter([1, 2, 3])
>>> it2 = iter([4, 5, 6])
>>> list(itertools.chain(it1, it2))  # chain
    returns an iterator
[1, 2, 3, 4, 5, 6]
```

In Python 3, `zip()` is built-in and already lazy (`itertools.izip` was removed):

```
>>> for x, y in zip(["a", "b", "c"], [1, 2, 3]):
...     print(x, y)
...
a 1
b 2
c 3
```

Quick Check: Generators

Think About It

`squares()` calls `integers()`, an infinite generator. Why doesn't `print(take(5, squares()))` hang forever?

Answer

Because generators are lazy: `integers()` doesn't produce any values until something asks for them via `next()`. `take()` only calls `next()` 5 times (via its `for i in range(n)` loop), so only 5 values ever get pulled through the whole chain – the "infinite" sequence is never actually materialized.